

Contents

OdinSTAR 2017 Workshop	13
Technical Web Testing and Automating From the Web Developer Tools	13
Alan Richardson	13
Basic Structure	13
Slides and Handouts and Links	14
Mix of Skills Supported	14
Content Overview	14
An Intro Demo	14
Z-Type	14
A Z-Type Bot	15
A Quick Demo of a Real App	15
Create Some Data in the App	15
Delete Them All	16
Demo Section	16
Hands off keyboards	16
Explanation and Demo of Basic Concepts	16
You Will See	16
About the Demo App	16
Every Browser Allows Us To	17
View Source	17
‘Inspect’ to View the DOM	17

‘Inspect’ to Amend the DOM	17
Other Dev Tools	18
JavaScript Console	18
HTML and JavaScript	18
changeSlogan	18
JavaScript Basics	19
Objects in Sloganizer	19
Functions	19
Some Functions in Sloganizer	20
Arrays	20
For Loops	20
Advanced Techniques	21
setInterval	21
Summary	21
JavaScript Summary	21
JavaScript ‘Bots’	22
JavaScript Sundries	22
Walkthrough Section	22
Hands on keyboards	22
Play Along At Home	22
Visit the Sloganizer site	23

Understand the Functionality	23
Understand the HTML	23
Why?	23
Inspect DOM with Dev tools	23
Why?	24
Manipulate DOM with Dev tools	24
Why?	24
View JavaScript Console	24
Why?	25
Use Sources to view JavaScript	25
Why?	25
Use Network to view traffic	25
Why?	25
Explore the Objects from console	26
View the source in Sources/Debugger/Resources	26
Wrap call to random_sentence in a for loop	26
Only 1, but we want to see 10	26
Why Run code from console?	27
Automate the App from the console	27
Why?	27

Any Questions?	27
About the games and challenges	28
Games and Challenges	28
This is your workshop - Ask Questions - Experiment	28
If you are not ready to move on yet	28
If current challenge is too basic for you	28
Console Driver	29
Console Driver	29
Console Driver Hints & Suggestions	29
Hints	29
Suggestions	29
Console Driver - Debrief	30
Console Driver - Some <i>Obvious</i> Secrets	30
Console Driver - Look at the source	30
Console Driver - Hack Targets	30
Console Driver - Console Hack Targets	31
Just Enough JavaScript To Be Dangerous - Bots	31
Just Enough JavaScript To Be Dangerous - for, if, array, break, \\, ++, ==	32
Just Enough JavaScript To Be Dangerous	32
Console Driver Bot Easy Suggestions	32
Console Driver Bot Challenges	33

Point Increaser Bot - Console Driver	33
Width Of Track Management Bot	33
Speed Bot - Console Driver	34
A Game Play Bot - Console Driver	34
Questions?	34
The Coloured Square Game	35
Play The Game	35
Debrief	35
Known Bugs	35
Quick Array Overview	36
DOM interaction with JavaScript Overview	36
You have to cheat to win	36
How many ways can you cheat to win?	36
Examples of ways to cheat - challenges	36
Examples of ways to cheat	37
Demo Amending Dom and Debugging if necessary	37
Use JavaScript in the console to set square to red	37
Use JavaScript in the console to set button to grey	37
Use JavaScript in the console to set button and square to same colour	38
Use JavaScript in the Console to change the contents of the array	38

Use Console to amend the checkColour function	38
Bonus Points in checkColour function	39
Questions?	39
The Coloured Square Changer Game	39
Play/Test The Game	39
Aim - Get a High Score however you can without amending score . .	39
Hints/Debrief	40
How Many Ways can you cheat?	40
Easy Examples	40
Other Examples	40
debugMode and colourBlindModeEnabled	41
enable and click on hidden changeLink link	41
Reduce Colours in Array	41
Negative Lives	41
Infinite Lives Bot	41
Re-enable changeColour	42
Colour Change Bot	42
Create a Bot to Play the Game	42
Questions?	43
Number Hover Text Game	43

Gameplay Demo	43
Number Hover Text Hints & Suggestions	43
Gameplay Hints	43
Suggestions	44
Number Hover Text - Debrief	44
Number Hover Text Game - Some <i>Obvious</i> Secrets	44
Number Hover Text Game - Look at the source	45
Number Hover Text - In Memory Hack Targets	45
Just Enough JavaScript To Be Dangerous - Bots	45
Just Enough JavaScript To Be Dangerous - JavaScript Objects	46
Number Hover Text Bot Suggestions & Challenges	46
DOM Manipulation Explored	46
In Memory Variable Manipulation	47
Boost Supply Bot	47
Magic Number Range Reset Bot	47
Never Zero Bot	48
Show Magic Number Bot	48
Gameplay Bot that ‘cheats’	49
Questions?	49
TodoMVC Application	49
How to start	50

Debrief	50
Explore The Code - Hard to understand if don't know backbone	50
Explore the Event Listeners from Developer View	51
Explore From the Console	51
Challenges	51
Create A Todo	51
Select a Todo	52
Delete a Todo	52
Amend a Todo	52
Exercises - Create, Toggle, Delete, Amend	53
Answer - use a for loop to create 100 todos	53
Answer - toggle all of your todos and mark them as completed	53
Answer - toggle all of todos to mark them as completed	54
Answer - write code to toggle half of them	54
Answer - Toggle Every Second Todo	54
Answer - Amend them all	55
Answer - Delete All Todos	55
Answer - Delete the completed items	55
Questions?	56
Bookmarklets	56

create 100 todos at console	56
Same code in a function allocated to t	56
Anonymous Function - Runs Immediately	57
As a Bookmarklet - run from URL	57
Delete All Todos as a Bookmarklet	57
Questions?	58
Recap DOM and Dev Tools	58
Recap JavaScript Capabilities	58
Recap on practice apps	58
Questions?	59
Final Section	59
Why? Because... Testing	59
Questions?	59
The End Of Workshop	59
Reference Section	59
Just Enough JavaScript To Be Dangerous	60
Console	60
Variables	60
Variables can hold pretty much any type of information e.g.	60
Operators	61

Strings	61
Integers	61
Hex	61
Math library/Object	62
Using the Console object	62
Alerts and Prompts	62
Functions	63
Functions	63
Iteration	63
for Loops	63
Arrays - Position Index	64
Arrays - Key Indexed	64
Arrays vs Objects	64
Selection and Conditions - if statements	65
Run Code Every X milliseconds	65
Interact with the DOM	66
Finding Elements	66
Change Style Element Attributes	66
Change Element Attributes	66
Change Element Text	67
Changing Event Code	67
Changing Event Code	67
Changing Event Code with Listeners	67

Removing Event Code Listeners	68
Bookmarklets	68
Build up code for bookmarklet	69
write out 100 lines to console	69
Same code in a function allocated to <code>t</code>	69
Anonymous Function - Bookmarklet	69
Anonymous Function - Runs Immediately	69
As a Bookmarklet - run from URL	69
Bookmarklet Resources From Alan	69
Bookmarklet Resources	70
XMLHttpRequest	70
More about XMLHttpRequest	70
Good JavaScript References For Future Study	71
Books	71
Tutorials	71
Good JavaScript References For Future Study	71
References	71
Links	71
EvilTester Sloganizer Application	71
Visit Experiment and Test	72
Debrief	72
From The Code	72
Exercises	73
Generate a random sentence using a sentences template	73

Generate 100 random sentences from a specific template	73
Generate 1000 random sentences	74
Generate a new slogan in the GUI every second	74
Hack the app to spit out your own slogans - using existing structure	74
Hack the app to spit out your own slogans - using new structure	74
Hack the app to spit out your own slogans - using new structure	75
I T.E.S.T With A.T.T.I.T.U.D.E app	75
Exercise	76
Debrief	76
Exercise: Investigation Hints	76
Debrief	77
More About?	77
Keep Going	77
ZType Bots	77
Crude Bot	77
Good Bot	78
A Z-Type Bot	78
BIO - Alan Richardson	78
Alan Richardson (www)	78
Follow	79

Books Written By Alan Richardson	79
Online Training Courses By Alan Richardson	79
The End	79

OdinSTAR 2017 Workshop

Technical Web Testing and Automating From the Web Developer Tools

Alan Richardson

“a combination of demonstrations, hands on exercises, and challenges to interact with, test and automate simple web applications, all from the browser”

- www.eviltester.com
 - twitter: [@eviltester](#)
-

Basic Structure

08:30 - 12:30

1st hour

- 08:30 - 08:40 General Overview (TELL)
 - Intro/Welcome
- 08:40 - 09:00 Me demo of Dev Tools and stuff (SHOW)
 - Demo/Explanations
- 09:00 - 09:30 Follow along Demo and Tutorial (DO)
 - Demo/Worked Examples

remainder

- 09:30 - 12:30 Guided Instructions (TELL / SHOW / DO)
 - split into 30 min chunks for different games and challenges & Debriefs
-

Slides and Handouts and Links

<http://compendiumdev.co.uk/page/atOdinStar2017>

- Also on the bottom of every slide.
-

Mix of Skills Supported

- I assume no knowledge
 - If the content is new to you then follow the demos and examples
 - If you already know a section then use the 'games list' to experiment
 - Please be considerate of others:
 - in a demo/lecture/debrief section no talking to neighbour,
 - do engage with the group
 - if you know something, share it with the group
 - if you learned something, share it with the group
 - Questions? Ask - someone else may want to know too
-

Content Overview

- learn how to tactically automate from Dev Tools console with JavaScript
 - learn how to observe, interrogate and manipulate the DOM
 - learn basics of JavaScript
 - learn basics of Dev Tools
-

An Intro Demo

Z-Type

- Z-Type online zty.pe
 - A typing Game
 - Bot is advanced
 - but at the end of this workshop you should understand it
 - you might need more practice to be able to write it
-

A Z-Type Bot

```
var zTypeBotAlphaChars = "abcdefghijklmnopqrstuvwxyz";
var zTypeBotCharToType = 0;
ztypebot = setInterval(function() {
    ig.game.shoot(
        zTypeBotAlphaChars.charAt(zTypeBotCharToType));
    zTypeBotCharToType = (zTypeBotCharToType + 1)%26;
}, 100);
```

Above bot is very crude but can play the game.

```
clearInterval(ztypebot);
```

A Quick Demo of a Real App

- Because this isn't just games
 - todomvc.com/examples/backbone
 - Demo to show how automating from the console supports us as a tactic for interactive and exploratory testing
-

Create Some Data in the App

```
for (x = 0; x < 100; x++) {
    app.todos.create({
        title: "todo ".concat(x),
        order: app.todos.nextOrder(),
        completed: false
    })
}
```

Delete Them All

```
for (x = app.todos.models.length - 1; x >= 0; x--) {  
  app.todos.models[x].destroy()  
}
```

Demo Section

Hands off keyboards

- Slide based demo section to provide an overview

Explanation and Demo of Basic Concepts

- We will have a hands on section after this
 - Do ask questions as we go through
-

You Will See

- Use of Browser Dev Tools
- HTML & DOM Interaction
- JavaScript Console Interaction
- Basics of JavaScript

After this, we will have a hands on guided walkthrough where you follow along.

About the Demo App

- The Evil Tester Sloganizer
 - <http://www.eviltester.com/apps/sloganizer/version/1/sloganizer.html>
 - Single Page App
 - Generates Random “Evil Tester” Slogans
 - Periodically shows an ‘ad’ for Book “Dear Evil Tester”
-

Every Browser Allows Us To

- View HTML Source
 - Manipulate DOM
 - Execute JavaScript
-

View Source

- Raw, Unformatted
 - The code sent from the server
 - Might have syntax highlighting
 - Might show syntax errors
-

‘Inspect’ to View the DOM

- View the DOM
 - Document Object Model
 - How the HTML is interpreted
 - See Styling
-

‘Inspect’ to Amend the DOM

- Anything in the browser can be amended
 - e.g. Change button text
 - Amend Applied CSS styles
 - e.g. “display:none”
 - Edit HTML Blocks
 - e.g. amend the ad
-

Other Dev Tools

- ‘Console’ view JavaScript errors, debug statements and execute JavaScript
 - ‘Sources’ view all pages and source files used
 - set breakpoints for JavaScript debugger
 - ‘Network’ view Network traffic
 - etc. Cookies, local storage
-

JavaScript Console

- Any JavaScript Errors shown here
 - Debug Messages shown here
 - Can also execute arbitrary JavaScript
 - `changeSlogan()`
-

HTML and JavaScript

- Inspect button

`onclick="changeSlogan()"`

- see also `Event Listeners`
 - see also `<script>` sections in page
 - see also any `.js` files in ‘Sources’ and ‘Network’
-

`changeSlogan`

- call with `()`
 - view without `()`
-

JavaScript Basics

- Objects
 - ‘things’ with properties and functions
 - Functions
 - things you can ‘call’ to do stuff
 - calls other functions
 - return values to calling functions
 - Variables
 - things with values you can change
 - `var myName = "Alan";`
-

Objects in Sloganizer

- an **object** called **phrases** which is a set of phrases, and these ‘phrases’ are arrays
- an **object** called **sentences** which is an array of **string**

Both of these objects (**phrases**, **sentences**) are assessible from the JavaScript console

If I type **phrases** at the console I will see information about that **object** in the console.

Functions

Functions look a bit like this:

```
function randomNumberUpTo(theVal){  
    return Math.floor(Math.random() * theVal)  
}
```

- This is a **function**
 - called **randomNumberUpTo**
 - it takes a single argument **theVal**
 - it uses the JavaScript **Math** library
 - it returns a **random** number up to the value of **theVal**
-

Some Functions in Sloganizer

- `random_sentence()`
- `process_sentence(sentence)`

Could call `random_sentence()` directly, need to pass an argument into `process_sentence`

```
process_sentence(sentences[0])
```

Arrays

The `sentences` object is an 'array'.

- `sentences[0]` would return the first item in the array
- `sentences.length` would return the length of the array
- `sentences[sentences.length-1]` would return the last value in the array

Can change values in arrays:

- `sentences[0] = "bob"` - change the first entry in the array to "bob"
-

For Loops

```
for(var x=1;x < 10; x++){  
    console.log(sentences[x]);  
}
```

- start x at 1
 - while x is less than 10
 - write the sentences array content at index x to the console `console.log`
 - increment x (`x++` means add 1 to x)
-

Advanced Techniques

setInterval

```
var sloganbot = setInterval(function(){
    changeSlogan()
},1000)
```

create a thread in variable `sloganbot` that calls `changeSlogan` every 1000 milliseconds.

Stop it with clearInterval

```
clearInterval(sloganbot)
```

Summary

- HTML and DOM can be different
 - Use Dev Tools to View and Amend DOM
 - Dev Tools let you view all sources loaded into browser
 - Dev Tools Console
 - lets you interact with JavaScript Interpreter
 - write code to interact with application
-

JavaScript Summary

- Objects, Functions, Arrays, Variables
 - all accessible and createable from console
 - for loops increment a variable until a condition is met
 - `for(var x=1;x < 10; x++){console.log(x);}`
 - call functions with `()` - add any arguments within `()`
-

JavaScript ‘Bots’

- `setInterval` to execute a function every X milliseconds (1000)

```
var mybot = setInterval(function(){  
    /* add the code you want in here */  
},1000)
```

- store your ‘bot’ interval in a variable so that you can stop it with

```
clearInterval(mybot);
```

JavaScript Sundries

- `var` to create a variable
- `[0]` access array values with `[]` first item starts at 0
- end statements with `;`
- comments `/* span lines or in lines */` and `// comment to end of line`

The Handouts contain a ‘Just Enough JavaScript To Be Dangerous’ section with more details of all of this.

Walkthrough Section

Hands on keyboards

Play Along At Home

- copy and paste from the document if you need to
 - you can’t break anything
 - ask questions if it isn’t clear
-

Visit the Sloganizer site

- The Evil Tester Sloganizer
 - <http://www.eviltester.com/apps/sloganizer/version/1/sloganizer.html>
 - or follow link from the workshop page
-

Understand the Functionality

- click the [I Want A Slogan] button a few times
-

Understand the HTML

- right click and view source
 - basics of HTML
 - sections: `head`, `body`, `style`, `script`
 - tags: `h1`, `h2`, `ul`, `li`, `div`
 - attributes: `class`, `href`, `onclick`, `id`, `style`
-

Why?

- learn HTML
 - review for data leaks
 - HTML comments might give more information about app
 - some source views show HTML syntax errors
-

Inspect DOM with Dev tools

- highlights position on page
- expand/collapse DOM elements

- see page structure
 - see styles in side menu
-

Why?

- understand structure of app
 - review for data leaks
 - review for accessibility
 - learn HTML
 - view CSS errors and issues
-

Manipulate DOM with Dev tools

- change text on the button
 - find the `bookinfo`
 - change class on the `bookinfo`
 - make `bookinfo` visible by changing style
 - delete the `bookinfo`
-

Why?

- put app in different states
 - amend values in forms to send data to server that might not be allowed
-

View JavaScript Console

- JavaScript errors now visible
 - refresh page
 - call the `onclick` function `changeSlogan()` from console
 - tab completion,
 - up and down to show command history
-

Why?

- errors and log messages are visible
 - use `console.log` to log values from the app
 - explore the app objects and data
-

Use Sources to view JavaScript

- Chrome (Sources), Edge/Firefox (Debugger), Safari (Resources)
 - You could set a breakpoint here
 - only mentioned for more advanced participants - not necessary for the exercises in this workshop
-

Why?

- Understand the application in more detail
 - Test at a lower level
 - Learn how to manipulate the app to support testing
 - Learn JavaScript
-

Use Network to view traffic

- Not necessary for exercises

Why?

- see what code is available
 - understand the ‘app’
 - any data leak issues?
-

Explore the Objects from console

- type `sentences`
 - should see it is an array
 - type `sentences[1]` gets the second item in the array
 - type `sentences.length` (property not a function call)
 - type `sentences[sentences.length-1]` (get the last item in the array)
-

View the source in Sources/Debugger/Resources

- look at the `changeSlogan` function code
 - it calls `random_sentence` to get a sentence
 - call `random_sentence` from console
 - type `random_sentence()`
-

Wrap call to `random_sentence` in a for loop

```
for(var s=0; s < 10; s++){  
    random_sentence();  
}
```

How many sentences did you see?

Only 1, but we want to see 10

- `random_sentence()` was called 10 times but only the last value was returned
- we actually want to `console.log` the output from each call

```
for(var s=0; s < 10; s++){  
    console.log(random_sentence());  
}
```

Tip: create a text file if it helps but you lose code completion, in Chrome use Snippets in Sources (right click run)(quick demo)

Why Run code from console?

- test smaller isolated parts of the application
 - setup data
 - put application in a specific state for testing
-

Automate the App from the console

```
var sloganbot = setInterval(function(){
    changeSlogan()
},1000)
```

create a thread in variable `sloganbot` that calls `changeSlogan` every 1000 milliseconds.

Stop it with `clearInterval`

```
clearInterval(sloganbot)
```

Why?

- longer running test
 - might expose JavaScript errors that appear over time
 - any memory leaks?
 - does it randomly generate slogans?
 - does the 'ad' appear?
 - Observe other parts of the app while it is being used
 - is there any network traffic?
 - monitor local storage and cookies?
-

Any Questions?

About the games and challenges

The next sections are:

- overview of game or challenge
 - play/automate game or take a challenge
 - hands on experimentation, I am here to help
 - group debrief and Q&A
 - some sections have demos and summaries
-

Games and Challenges

Designed to give you the chance to explore:

- Dev Tools
 - DOM Manipulation
 - JavaScript Console
 - JavaScript Techniques
-

This is your workshop - Ask Questions - Experiment

If you are not ready to move on yet

- continue working through the challenges and tasks
- read the slides: learn JavaScript - try out the code examples

If current challenge is too basic for you

- solve it then move on to other challenges and games on the list of games or slides <http://compendiumdev.co.uk/games/buggygames/>
 - read the slides, experiment with JavaScript you don't know
 - **continue to join in with debriefs, demos, and Q&A sessions**
-

Console Driver

- a Basic Game
 - 5 mins play/test/explore it session
 - debrief
-

Console Driver

The Ultimate Driving Game Experience. You are a road warrior hell bent on speeding along the ultimate winding road. Can you survive longer than anyone else?

- Thrill to the ever increasing speed of travel
- Gasp as you narrowly avoid hitting the lethal road wall
- Put your driving skills to the test with trigger fast controls

Don't Die.

Play Console Driver from compendiumdev.co.uk/games/buggygames

Console Driver Hints & Suggestions

Hints

- Use the Browser Dev Tools Console
- Have Focus on the Page to play the Game

Suggestions

- Are there any bugs?
- Have you looked at the code?
- Can you cheat?
 - can you score more than “291551004004002940”?

Play Console Driver from compendiumdev.co.uk/games/buggygames

Console Driver - Debrief

- Knowledge Sharing
 - What did you do?
 - How did you do it?
 - What problems did you find in the game?
 - What problems did you have testing it?
 - Did you keep notes?
 - What tools did you use?
-

Console Driver - Some *Obvious* Secrets

- infinite game bugs
 - go right very quickly when there are still gaps in wall for infinite game
 - go left very quickly when there are still gaps in wall for invisible infinite game
 - cannot amend `points` but can amend `distanceTravelled` which is the basis for `points`
 - did you find and try the `setLevel` GUI?
 - did you `promptForValues=true`?
 - Hint - press ENTER key before starting game.
-

Console Driver - Look at the source

- `document.onkeydown` is important event handler
 - use the source to learn how to write code that manipulates the game e.g. cut and paste out from source into console, change variables, etc.
-

Console Driver - Hack Targets

- `distanceTravelled`
 - controls the score which is displayed as `points`

- `widthOfTrack`
 - controls the width of the track

All of these could be amended at the console.

e.g. `variableName = value`

Console Driver - Console Hack Targets

Just Enough JavaScript To Be Dangerous

Use the Console to amend variables

e.g.

- `widthOfTrack=30`
 - `distanceTravelled+=100` //(add 100 to `distanceTravelled`)
-

Just Enough JavaScript To Be Dangerous - Bots

- To 'Create' A Bot that runs every 100 milliseconds

```
var myBot = setInterval(function() {  
    //whatever JavaScript Code I want in here;  
}, 100);
```

- To 'Kill' the Bot

```
clearInterval(myBot);
```

Just Enough JavaScript To Be Dangerous - for, if, array, break, \, ++, ==

```
for (i = 0; i < anArray.length; i++) {  
    if(anArray[i]==" "){  
        // do something;  
        break;  
    }  
}
```

Just Enough JavaScript To Be Dangerous

- for loop executes code within {} until a condition
- *for(setup; until true; at end of each iteration)*
- arrays have numeric keys and start at 0
 - e.g. `anArray[0]` is first entry in an Array
- `i++` increment value of variable i by 1
- arrays have a 'length'
- `break` exit a loop early
- *if(condition is true){ then do stuff in here }*

Console Driver Bot Easy Suggestions

Can you write the following Bots?

- Point Increaser Bot
 - every 500 milliseconds increase `distanceTravelled` for a subtle points boost
- Width Of Track Management Bot
 - every 500 milliseconds the bot maintains the width of the track for you

Combine Bots and/or Cheats for Mega Points

Console Driver Bot Challenges

Can you write more advanced Bots?

- Speed Brake Bot
 - every 500 milliseconds the Bot sets the speed to a manageable speed for you
 - Create a Bot that plays the game
 - No hints - look at the code and figure it out
-

Point Increaser Bot - Console Driver

```
var myPointsBot = setInterval(function() {  
    distanceTravelled=distanceTravelled+50;  
}, 500);
```

and

```
clearInterval(myPointsBot);
```

Width Of Track Management Bot

```
var myWidthBot = setInterval(function() {  
    widthOfTrack=58;  
}, 500);
```

and

```
clearInterval(myWidthBot);
```

Speed Bot - Console Driver

Cannot just change `millisecondsBetweenScrolls`, need to amend difficulty levels.

- set to lowest level (0) for slow speed, or fastest level (6)

```
var mySpeedBot = setInterval(function() {  
  setDifficultyLevel(0);  
}, 500);
```

and

```
clearInterval(mySpeedBot);
```

A Game Play Bot - Console Driver

```
var myCarControlBot = setInterval(function() {  
  // using code from `displayTheTrackToTheConsole`  
  var trackDisplay = getTrackLine(posOfLeftOfTrack,  
    widthOfTrack, maxWidthOfArena);  
  var trackArray = trackDisplay.split("");  
  var carSpace = trackArray[carPosition];  
  for (i = 0; i < trackArray.length; i++) {  
    if(trackArray[i]==" " && trackArray[i+1]==" "){  
      carPosition=i+1;  
      break;  
    }  
  }  
}, 50);  
  
clearInterval(myCarControlBot);
```

Questions?

The Coloured Square Game

Click the link to change the colour, when it is the colour you want then press the button.

The Ultimate Game of colour matching and cheating.

How many ways can you cheat to win?

- compendiumdev.co.uk/games/buggygames
 - /the_coloured_square_game/colouredsquare.html
-

Play The Game

- Play The Game
- Look at the Code
- Any Bugs?

5 Minutes

Debrief

- What did we learn?
-

Known Bugs

- square starts as grey but never turns back to grey
- it never turns “red”
 - “red” is not in the array of colours

```
var colours = ["blue", "white", "green", "black", "pink"];
```

Quick Array Overview

- View array at console
 - `colours`
 - create an empty array
 - `colours = []`;
 - set a value in array
 - `colours[0] = "green"`;
-

DOM interaction with JavaScript Overview

- find an element by id
 - `document.getElementById("changeLink")`
- get styles etc.
 - `element.style.backgroundColor`
- can change styles etc. = `.style.backgroundColor = "black"`
- `getAttribute`, `setAttribute`, `id`, `innerHTML` etc.

Look in the code and see how the game does it.

You have to cheat to win

How many ways can you cheat to win?

Examples of ways to cheat - challenges

- Amend Dom to show winning and show alert then take screenshot
 - Amend the colour of the button to match the square with dev tools
 - Amend the colour of the square to match the button with dev tools
 - Amend the array to include “red” during a breakpoint debug mode
 - Use Debugger to change running code variables
-

Examples of ways to cheat

- Use javascript in the console to
 - set square colour and press the button
 - set button colour to match square and press the button
 - set button and square to same colour
 - change the contents of the array
 - change the `checkColour` function
- If running from a server then use proxy to amend code prior to loading
 - Amend the code to remove the check or change it to `'if(true)'`

Demo Amending Dom and Debugging if necessary

Use JavaScript in the console to set square to red

```
document.getElementById("theSquare").  
  style.backgroundColor = "red";
```

Use JavaScript in the console to set button to grey

When the game starts, the square is **grey** so we could copy the default in the code and set the button to match.

```
document.getElementById("theButton").  
  style.backgroundColor = "grey";
```

Use JavaScript in the console to set button and square to same colour

```
document.getElementById("theSquare").  
  style.backgroundColor =  
    document.getElementById("theButton").  
      style.backgroundColor;
```

Use JavaScript in the Console to change the contents of the array

```
var colours = ["red", "red", "red", "red", "red"];
```

or even just

```
var colours = ["red"];
```

Use Console to amend the checkColour function

```
checkColour = function(){  
  changeStatusText("Winner!");  
  alert("You Win!");  
}
```

For bonus points:

- Change the colour of the square
 - Change the link text to say "Change From red"
-

Bonus Points in checkColour function

```
checkColour = function(){  
    document.getElementById("theSquare").  
        style.backgroundColor = "red";  
    changeLinkText("Change From red")  
    changeStatusText("Winner!");  
    alert("You Win!");  
}
```

Questions?

The Coloured Square Changer Game

A variant of the Coloured Square Game.

- Marvel as it changes colour on its own
- Can you get a high score by matching the colours?

compendiumdev.co.uk/games/buggygames/the_coloured_square_game/colouredsquarechanger.html

Play/Test The Game

Aim - Get a High Score however you can without amending score

- What did you score?
 - What did you notice?
-

Hints/Debrief

- Did you find the hidden link?
 - Does `colourBlindModeEnabled` help?
 - Does `debugMode` help?
 - How can you take advantage of the missing `changeColour` function?
-

How Many Ways can you cheat?

Without amending `score`?

Easy Examples

DOM

- enable and click on hidden `changeLink` link

JavaScript

- set `debugMode` variable to `true`
 - set `colourBlindModeEnabled` variable to `true`
 - reduce colours in array
 - negative lives
 - set timing variables to make game easier
-

Other Examples

- Infinite Lives Bot
 - Re-enable `changeColour`
 - Colour Change Bot
 - Can you Create a Bot to Play the Game
-

debugMode and colourBlindModeEnabled

Give more information when playing. Which might help.

enable and click on hidden changeLink link

Use Dev tools to remove `display:none` then click on link to change colour when you want rather than wait on timer.

Reduce Colours in Array

```
colours = ["red"];
```

Makes the game a lot easier.

Negative Lives

The end of game check is for `lives===0` so if lives somehow goes **-ve** then the game never ends.

```
lives=-1;
```

Infinite Lives Bot

```
var infiniteLivesBot = setInterval(function(){  
    if(lives<2){lives=4;}  
},5);
```

```
clearInterval(infiniteLivesBot);
```

Re-enable changeColour

```
changeColour = function(){
    document.getElementById("theSquare")
        .style.backgroundColor=
        document.getElementById("theButton").
            style.backgroundColor;
}
```

Colour Change Bot

```
var colourChangeBot = setInterval(function(){
    var sq = document.getElementById("theSquare");
    var bt = document.getElementById("theButton");

    if(sq.style.backgroundColor!=
        bt.style.backgroundColor){
        sq.style.backgroundColor=
            bt.style.backgroundColor;}
},500);

clearInterval(colourChangeBot);
```

Create a Bot to Play the Game

```
var gamePlayBot = setInterval(function(){
    var sq = document.getElementById("theSquare");
    var bt = document.getElementById("theButton");

    if(sq.style.backgroundColor===
        bt.style.backgroundColor){
        checkColour();}
},100);

clearInterval(gamePlayBot);
```

To help it get started

```
timeToChangeSquare=500; timeToChangeButton=500;  
currentButtonChanges=10; currentSquareChanges=10;
```

Questions?

Number Hover Text Game

The Ultimate Jet Pack Hover Experience. You are a jet pack test pilot that must hover in certain positions to gain points. But no-one told you what those positions are, so boost to adjust your height and when you are in the correct position you will score points. Beware you have limited boost but your boost will reset at the same time as the correct position. Can you score more than anyone else?

- Thrill to the updating text dashboard
- Gasp as you run out of boost seconds before reset
- Groan as you realise you wasted your boost

Don't Hit Zero.

Gameplay Demo

“Number Hover Text Game” from compendiumdev.co.uk/games/buggygames

Number Hover Text Hints & Suggestions

Gameplay Hints

- Watch the Points and when they change you have found the magic number
- Tap The Up Key To Hover The Points

- When the Points Change, hold down up to boost to the top
 - Don't hold down up when count is at 99 otherwise you will waste your boost
-

Suggestions

- Look at the code
- Have you amended the DOM with Inspect Browser Dev Tools?
- What does debug mode do?
- There are so many ways to cheat at this game - can you get a high score without amending the `points` score variable?

Play/Hack/Test “Number Hover Text Game” from compendiumdev.co.uk/games/buggygames

Number Hover Text - Debrief

- Knowledge Sharing
 - What did you do?
 - How did you do it?
 - What problems did you find in the game?
 - What problems did you have testing it?
 - Did you keep notes?
 - What tools did you use?
-

Number Hover Text Game - Some *Obvious* Secrets

- Debug Mode is accessible through Dom Manipulation
-

Number Hover Text Game - Look at the source

Number Hover Text - In Memory Hack Targets

- `theCount`
 - the current count down value
- `magicNumber`
 - the number to hover around (you could set it to a known value)
- `boost`
 - to increase your boost
- `boostCost`
 - you could have infinite boost

All of these can be amended at the console.

Just Enough JavaScript To Be Dangerous - Bots

- To 'Create' A Bot that runs every 100 milliseconds

```
var myBot = setInterval(function() {  
    //whatever JavaScript Code I want in here;  
}, 100);
```

- To 'Kill' the Bot

```
clearInterval(myBot);
```

Just Enough JavaScript To Be Dangerous - JavaScript Objects

```
var aNewObject = {};  
aNewObject.property = "bob";
```

Just Enough JavaScript To Be Dangerous - creates a new object called `aNewObject` - adds a field called `property` to the object and sets it to `"bob"` - any code that accesses `aNewObject.property` would find the value `"bob"`

Number Hover Text Bot Suggestions & Challenges

- Create a Bot that supplies more boost when you get low
- Create a Bot that prevents you moving too far away from the Magic Number
- Create a Bot that prevents you hitting zero
- Create a Bot that displays the Magic Number on screen
- Create a Bot that plays the game without needing any other bots
- Create a Bot that plays the game without ‘cheating’

Combine Bots and/or Cheats for Mega Points

DOM Manipulation Explored

On the Instructions Page, if we inspect element and look at the `div` with `id` of `defaults` then we can amend these values to configure the game.

- set `debugMode` to 1 then the game uses the values in this section as defaults
- e.g.
 - give yourself more boost to make the game easier
 - increase ‘points change after’ to give you more time to hover over the magic number

In Memory Variable Manipulation

Using the console it is possible to amend many of the variables as the game plays e.g.

- `theCount=magicNumber+2`
 - would continue to reset `theCount` to 'just above' the magic number so when the count down decrements automatically you will score points.
-

Boost Supply Bot

- Create a Bot that supplies more boost when you get low

```
var myBoostSupplyBot = setInterval(function() {  
    if(boost<10){boost=1000;}  
}, 100);
```

- To 'Kill' the Bot

```
clearInterval(myBoostSupplyBot);
```

Magic Number Range Reset Bot

- Create a Bot that prevents you moving too far away from the Magic Number

```
var myMagicNumberRangeBot = setInterval(function() {  
    var withinRange=5;  
    if(theCount<magicNumber-withinRange){  
        var newCount = magicNumber-withinRange;  
        // prevent death  
        if(newCount<1){  
            newCount=1;  
        }  
        theCount= newCount;  
    }  
}, 100);
```

- To ‘Kill’ the Bot

```
clearInterval(myMagicNumberRangeBot);
```

Never Zero Bot

- Create a Bot that prevents you hitting zero

```
var myNeverZeroBot = setInterval(function() {  
    if(theCount<1){  
        theCount=1;  
    }  
}, 10);
```

- To ‘Kill’ the Bot

```
clearInterval(myNeverZeroBot);
```

Show Magic Number Bot

The JavaScript code has some useful functions that we could re-use for our own purposes. e.g. `changeSpanText` which the game uses to show stats.

I could setup a bot to show the Magic Number `magicNumber` instead of the Boost.

```
var myMagicNumberBot = setInterval(function() {  
    changeSpanText("boost",magicNumber);  
}, 100);
```

- To ‘Kill’ the Bot

```
clearInterval(myMagicNumberBot);
```

Gameplay Bot that ‘cheats’

- Create a Bot that plays the game without any other bots
- It ‘cheats’ because it ‘knows’ what the magicNumber is

```
var myFirstGameplayBot = setInterval(function() {  
    if(theCount<magicNumber){  
        var myFakeEvent = {};  
        myFakeEvent.keyCode=38;  
        document.onkeydown(myFakeEvent);  
    }  
}, 100);
```

- To ‘Kill’ the Bot

```
clearInterval(myFirstGameplayBot);
```

Questions?

TodoMVC Application

It is another single page application. But it is complicated by the fact that it uses a JavaScript framework.

Unfortunately that means we really need to learn the technology that it is built on in order to test it properly.

- BackboneJS.org

But even if we don’t know the framework we can still interact with the application to help us with our testing.

- TodoMVC.com
 - todomvc.com/examples/backbone
-

How to start

- Use the App

then

- Look at the code

5 Minutes

Debrief

- What did we learn?
-

Explore The Code - Hard to understand if don't know backbone

- `examples/backbone` the main code folder
- `/js`
 - `collections` - the instantiated 'models'
 - `models` - probably the domain model
 - `routers` - how it is wired together
 - `views` - how things are displayed
- `collections/todos.js`

Top level objects:

```
var app = app || {};  
app.todos = new Todos();
```

Explore the Event Listeners from Developer View

Easier to understand because contextual.

- Jump to code in app-view.js
 - can see roughly how to create
 - can see how to loop around with `.each`
-

Explore From the Console

```
> app
< Object {todos: child, TodoRouter: child, TodoFilter: ""}

> app.todos
< child {length: 2, models: Array[2], _byId: Object
  , _listenId: "12", _events: Object}
```

Challenges

Use the console to:

- create todos (see walkthrough on next slide)
 - select todos
 - delete todos
 - amend todos
-

Create A Todo

Using information from 'find' on `examples/backbone'` for "create" in `app-view.js`.

```
app.todos.create(this.newAttributes());
```

- Examine the `newAttributes` function and learn:

```
app.todos.create({title:"auto todo",  
                 order: app.todos.nextOrder(),  
                 completed: false})
```

Select a Todo

Look in `todo.js`

```
app.todos.models[0].toggle()
```

Delete a Todo

```
app.todos.models[0].destroy()
```

Amend a Todo

The clue for amendment is in the `todo.js`. The `toggle` function uses a `save` function, which takes an object with the properties we want to amend:

```
app.todos.models[0].save({title:"bob"})
```

Amend existing details:

```
app.todos.models[0].save(  
  {title: app.todos.models[0].get("title").concat("X")}  
)
```

Exercises - Create, Toggle, Delete, Amend

- use a `for` loop to create 100 todos
 - use a `for` loop to `toggle` all of your todos and mark them as `completed`, then use the GUI to `Clear completed`
 - create another 100 , then write code to toggle half of them
 - create another 100, write code to toggle every second todo
 - create another 100 todos, then amend them all to have “*TODO*” after the name, e.g. “todo” would become “todo*TODO*”
 - create another 100 todos, then delete them all
 - create another 100, complete half, delete all completed items
-

Answer - use a `for` loop to create 100 todos

- use a `for` loop to create 100 todos

```
for (x = 0; x < 100; x++) {  
  app.todos.create({  
    title: "todo ".concat(x),  
    order: app.todos.nextOrder(),  
    completed: false  
  })  
}
```

Or title: "todo " + x,

Answer - `toggle` all of your todos and mark them as `completed`

- use a `for` loop to `toggle` all of your todos and mark them as `completed`, then use the GUI to `Clear completed`

```
for (x = 0; x < app.todos.models.length; x++) {  
  app.todos.models[x].toggle()  
}
```

Answer - toggle all of todos to mark them as completed

```
app.todos.each(function (todo) {
    todo.toggle();
});
```

Code found by using 'inspect' and 'event listeners'

Answer - write code to toggle half of them

- create another 100 todos, then write code to toggle half of them

```
for (x = 0; x < (app.todos.models.length / 2); x++) {
    app.todos.models[x].toggle()
}
```

Answer - Toggle Every Second Todo

- create another 100 todos, then write code to toggle every second todo

```
for (x = 0; x < app.todos.models.length; x++) {
    if (x % 2 === 1) {
        app.todos.models[x].toggle()
    }
}
```

Or using a variable

```
var toggleThisOne = false;
for (x = 0; x < app.todos.models.length; x++) {
    if (toggleThisOne) {
        app.todos.models[x].toggle()
    }
    toggleThisOne = !toggleThisOne;
}
```

Answer - Amend them all

- create another 100 todos, then amend them all to have “*TODO*” after the name, e.g. “todo” would become “todo*TODO*”

```
for (x = 0; x < app.todos.models.length; x++) {  
  app.todos.models[x].save({  
    title: app.todos.models[x].get("title").concat("*TODO*")  
  })  
}
```

Answer - Delete All Todos

- create another 100 todos, then see if you can delete them all

```
for (x = app.todos.models.length - 1; x >= 0; x--) {  
  app.todos.models[x].destroy()  
}
```

Need to work backwards through the array as it is changing as we delete.

Answer - Delete the completed items

- create another 100 todos, complete half of them, then delete the completed items

The `completed()` function returns an array each time it is called.

```
while (app.todos.completed().length > 0) {  
  app.todos.completed()[0].destroy();  
}
```

Questions?

Bookmarklets

Any JavaScript code can be:

- wrapped in a function and called
- wrapped in a function and run immediately
- wrapped in a function and called as a url and stored as a bookmarklet.

e.g.

- worked example using create 100 todos at todomvc.com/examples/backbone
-

create 100 todos at console

```
for(x = 0; x < 100; x++) {  
  app.todos.create({  
    title: "todo ".concat(x),  
    order: app.todos.nextOrder(),  
    completed: false  
  })  
}
```

Same code in a function allocated to t

```
t = function(){  
  for(x = 0; x < 100; x++) {  
    app.todos.create({  
      title: "todo ".concat(x),  
      order: app.todos.nextOrder(),  
      completed: false  
    })  
  }  
}
```

run `t()` to create 100 todos

Anonymous Function - Runs Immediately

```
(function(){
  for(x = 0; x < 100; x++) {
    app.todos.create({
      title: "todo ".concat(x),
      order: app.todos.nextOrder(),
      completed: false
    })
  }
})();
```

As a Bookmarklet - run from URL

```
javascript:(function(){
  for(x = 0; x < 100; x++) {
    app.todos.create({
      title: "todo ".concat(x),
      order: app.todos.nextOrder(),
      completed: false
    })
  }
})();
```

Delete All Todos as a Bookmarklet

```
for(x = app.todos.models.length - 1; x >= 0; x--) {
  app.todos.models[x].destroy()
}

(function(){
  for(x = app.todos.models.length - 1; x >= 0; x--) {
    app.todos.models[x].destroy()
  })()
```

```
javascript:(function(){  
for(x = app.todos.models.length - 1; x >= 0; x--) {  
    app.todos.models[x].destroy()  
}})()
```

Questions?

Recap DOM and Dev Tools

- DOM Investigation with inspect
 - Can Amend DOM
 - Events link to code in Event Listener
 - Code Exploration in Sources
 - code snippets in Chrome
-

Recap JavaScript Capabilities

- Amend variables from console
 - Run code from console
 - `for` loops block processing
 - use `setInterval` bots for automating in parallel
 - bookmarklets to run code samples
-

Recap on practice apps

There are other games and apps listed here:

compendiumdev.co.uk/games/buggygames

See also reference section.

Questions?

Final Section

- You should be familiar with the dev tools now
 - Learn to read HTML
 - Learn to manipulate HTML with dev tools
 - Learn to manipulate HTML with JavaScript
-

Why? Because... Testing

- Anything your app can do, you can do
 - This is tactical automation
 - You don't need to know much JavaScript to achieve a lot
-

Questions?

The End Of Workshop

- www.compendiumdev.co.uk
-

Reference Section

The following section is unlikely to be covered in the workshop.

It contains a reference on the JavaScript covered throughout the workshop - but in more detail.

Also Additional Games and Challenges.

Just Enough JavaScript To Be Dangerous

This section will provide a short reference to JavaScript. Not all of JavaScript you understand, just enough to be dangerous!

Console

Right click on your web page to access the developer tools.

Click on the 'Console' tab and you can start writing JavaScript.

Variables

```
var debugMode=true;
```

- The variable has a 'name' i.e. `debugMode`
 - we could 'declare' it by missing out the `=true`
 - we are initializing it (setting its value for the first time) with `=true`
 - Add the ';' at the end of the line
 - The variable has a **value** i.e. `true`
 - we can type the variable name into the console to see information about it
 - we assign it a new value with `=` e.g. `=false`
-

Variables can hold pretty much any type of information e.g.

- boolean : `true`, `false`
 - String : `"Bob"`
 - Integer : `'23'`
 - functions
 - objects
-

Operators

`--/+*`

- `++` to increment a variable
 - `--` to decrement a variable
 - `+=` to add a value e.g. `x+=5` (also `/=`, `*=`, `-=`)
 - `'+'` can be used to add numbers and concatenate strings
-

Strings

Strings can be `"` or `'`

- `substr(...)` - get sub string
 - `.substr(-2)` - the last 2 characters
 - `.substr(2)` - the string starting from the 2nd character
 - `.substr(2,8)` - the string from chars 2 to 8
 - `split(...)` - create an array by splitting based on a char
 - `.split(",")`
 - `parseInt("10")` - takes a string and returns an integer, whitespace is ignored, NaN returned if not a convertible String
 - developer.mozilla.org/en/docs/Web/JavaScript/Reference/Global_Objects/parseInt
 - `.length` - length of string
 - `.concat(...)` - return string with argument ... concatenated
-

Integers

Hex

- `myNum.toString(16)` - convert to hex
- `var myNum = 0xFFFFF;` - initialize a variable using HEX notation

Math library/Object

JavaScript has a standard Math library/Object to do math stuff

- `Math.random()`

w3schools.com/js/js_math.asp

Using the Console object

If I type `debugMode` into the console I will see the value.

But I can also print information to the console with `console.log`

e.g.

- `console.log('Hello');`
 - `console.log(debugMode);`
-

Alerts and Prompts

We can make pop up dialogs appear.

```
alert("Hello");
```

```
prompt("type your name", 'no name');
```

- The second argument is optional
- `prompt` returns what the user types in

But this stops other JavaScript from running until the alert has been dismissed.

Note `"` and `'` are pretty much interchangeable.

Functions

Functions let us create code that we can call at any time we want.

```
function randomNumberUpTo(theVal){
    return Math.floor(Math.random() * theVal)
}
randomNumberUpTo(10;)
```

- This is a function
 - called `randomNumberUpTo`
 - it takes a single argument `theVal`
 - it uses the JavaScript `Math` library
 - it returns a random number up to the value of `theVal`
-

Functions

We can allocate functions to variables.

```
var randomNum = function randomNumberUpTo(theVal){
    return Math.floor(Math.random() * theVal)
};
randomNum(10);
```

Iteration

for Loops

```
for(x=0;x<100;x++){
    console.log("line " + x);
}
```

- loop from 0 to 99 ($x < 100$)
 - increment by 1 each time ($x++$)
 - each time run the code within `{}`
 - log to the console `"line "` with `x` appended e.g. `'line 0'`, `'line 1'`
-

Arrays - Position Index

```
var colours = ["red", "green", "blue", "grey", "black"];
```

- An array that is indexed by position
 - `colours[0]=== "red"` arrays start at index 0
 - `colours[5]===undefined` last index in the array is 4
 - Arrays have length
 - `colours.length===5`
 - The last index in an array is `length-1`
 - `colours[colours.length-1]=== "black"`
 - We can set the values in the array
 - `colours[0] = "white";` the first item is now "white"
 - `.join(",")` - create a string by concatenating all elements together with ","
-

Arrays - Key Indexed

Arrays can be 'key' indexed;

```
var colours = [];  
colours["stop"]="red";
```

Then `colours["stop"]=== "red"` is true.

Arrays vs Objects

Arrays and Objects are very similar

```
var colours = {"stop": "red"};
```

Then `colours["stop"]=== "red"` is also true.

- Objects {}
 - Arrays []
-

Selection and Conditions - if statements

```
var debugMode=true;
if(debugMode===true){
    console.log("debug mode is on");
}else{
    console.log("debug mode is not on");
}
```

JavaScript has strange equality operators:

- === check if something equals
-

Run Code Every X milliseconds

`setInterval` runs a function every 'X' milliseconds

```
var sloganbot = setInterval(changeSlogan,1000)
```

- every 1000 milliseconds run the function `changeSlogan`

```
var sloganbot = setInterval(function(){changeSlogan()},1000)
```

- every 1000 milliseconds run the 'anonymous' function
 - in this case the 'anonymous' function calls the `changeSlogan` function
- You can add as much code as you want in an 'anonymous' function

Stop an 'interval' with `clearInterval`

```
clearInterval(sloganbot)
```

Interact with the DOM

Finding Elements

- `getElementById`
- `getElementsByClassName`
- `getElementsByName`
- `getElementsByTagName`
- `querySelector`
- use CSS selector as the query argument to match first element
- `querySelectorAll`
- return all in an array using CSS selector

can find from an element or document

Change Style Element Attributes

```
document.getElementById("theSquare").  
    style.backgroundColor = "red";
```

- get individual elements with `getElementById`
- access the `style` attribute of an element with `.style`

Hiding an element:

```
document.getElementById("theSquare").style.display="none"
```

- set to “block” or “” to display it

Change Element Attributes

```
document.getElementsByClassName("title")[0].  
    setAttribute("style","display:none");
```

```
document.getElementsByClassName("title")[0].  
    getAttribute("style");
```

Change Element Text

```
elem.innerHTML = "Hello";
```

Changing Event Code

The system might have defined events.

```
<div class="title">
  <button class="title" onclick="changeSlogan()">
    I Want A Slogan
  </button>
</div>
```

We can change them.

Changing Event Code

```
onclick="changeSlogan()"

var theButton=document.getElementsByClassName("title")[2];
theButton.onclick = function() {
    alert('hello');
};
```

Changing Event Code with Listeners

```
var theButton=document.getElementsByClassName("title")[2];
theButton.onclick = null;
theButton.addEventListener("click", function(){
    alert('from adding event');
}, false);
```

- The `onclick` attribute will operate alongside event handlers.

- `false` is for 'useCapture' - true for capturing phase, false (default) for bubbling phase
 - bubbling for handling child events
 - capturing - has to be element clicked on
 - 'useCapture' is optional
-

Removing Event Code Listeners

Create function into a variable if we want to remove the event Listener

```
var theButton=document.getElementsByClassName("title")[2];
theButton.onclick = null;
var eventHandlingFunction =
    function(){alert('from adding event');}
theButton.addEventListener("click",
    eventHandlingFunction, false);
```

Same phase must be specified to remove it.

```
theButton.removeEventListener("click",
    eventHandlingFunction, false);
```

Bookmarklets

Any JavaScript code can be:

- wrapped in a function and called
- wrapped in a function and run immediately
- wrapped in a function and called as a url and stored as a bookmarklet.

e.g.

- worked example
-

Build up code for bookmarklet

write out 100 lines to console

```
for(x = 0; x < 100; x++) {console.log(x)}
```

Same code in a function allocated to **t**

```
t = function(){  
    for(x = 0; x < 100; x++) {console.log(x)}  
}
```

run **t()** to write 100 lines

Anonymous Function - Bookmarklet

Anonymous Function - Runs Immediately

```
(function(){  
    for(x = 0; x < 100; x++) {console.log(x)}  
})();
```

As a Bookmarklet - run from URL

```
javascript:(function(){  
    for(x = 0; x < 100; x++) {console.log(x)}  
})();
```

Bookmarklet Resources From Alan

- a Tool
 - <http://compendiumdev.co.uk/apps/eviltester/bookmarklets.html>
 - <https://github.com/eviltester/bookmarklet>
 - Blog Post
 - <http://blog.eviltester.com/2017/04/a-quick-intro-to-bookmarklets.html>
-

Bookmarklet Resources

Resources:

- <http://adrianroselli.com/2015/01/css-bookmarklets-for-testing-and-fixing.html>
 - <http://www.dev-hq.net/posts/1-create-javascript-bookmarklet>
 - <http://www.squarefree.com/userstyles/make-bookmarklet.html>
 - <https://www.smashingmagazine.com/2010/05/make-your-own-bookmarklets-with-jquery/>
-

XMLHttpRequest

XMLHttpRequest is JavaScript's way of making HTTP requests.

```
xmlhttp = new XMLHttpRequest();  
xmlhttp.open("GET", "/player/user/open/e", false);  
xmlhttp.send(null);
```

The above:

- creates an `XMLHttpRequest` object
 - `open` a GET request on url `/player/user/open/e` (`false` means it is a synchronous request)
 - `send` the request with a `null` body because it is a GET request
-

More about XMLHttpRequest

You can find more details on `XMLHttpRequest` online

w3schools.com/xml/ajax_xmlhttprequest_create.asp

This is very handy if your API is on the same domain as the web site and there are no cross domain issues.

Also if you know the URL structure of an application then you can hit any Forms or GET requests (if they have side-effects).

Good JavaScript References For Future Study

Books

- [Eloquent JavaScript](#)
- [JavaScript for Cats](#)

Tutorials

- [\[w3schools.com/js\]](http://www.w3schools.com/js/)<http://www.w3schools.com/js/>
 - [Mozilla Developer Network Javascript](#)
 - htmldog.com/guides/javascript
 - learn-js.org
-

Good JavaScript References For Future Study

References

- w3schools.com/jsref

Links

- [Essential JavaScript Links](#)
-

EvilTester Sloganizer Application

A single page application. That generates random slogans related to software testing.

It may well have bugs.

- Live Version EvilTester.com/apps/sloganizer.html
 - Version 1 EvilTester.com/apps/sloganizer/version/1/sloganizer.html
 - Source github.com/eviltester/sloganizer
-

Visit Experiment and Test

- Can you find any bugs in version 1?
 - Version 1 EvilTester.com/apps/sloganizer/version/1/sloganizer.html
 - Look at the Code
 - What can you make it do?
 - Any differences between early version and live version?
 - Live Version EvilTester.com/apps/sloganizer.html
-

Debrief

- What did we learn?
-

From The Code

Main Entry Point

```
<div class="title">
  <button class="title" onclick="changeSlogan()">
    I Want A Slogan
  </button>
</div>
```

Top Level Objects:

- phrases
 - sentences
-

Exercises

- Look at the source for the sloganizer and see how much you understand
 - type the names of the objects that you see into the console
 - explore the output in the console
 - run some of the functions that you find
 - Generate a random sentence using a `sentences` template
 - `process_sentence` with an item from the `sentences` array
 - Generate 100 random sentences from a specific template
 - Generate 1000 random sentences
 - Generate a new slogan in the GUI every second
 - Hack the app to spit out your own slogans
-

Generate a random sentence using a `sentences` template

```
process_sentence(sentences[0]);
```

Can you see the difference between these approaches?

```
console.log(process_sentence(sentences[0]));
```

Generate 100 random sentences from a specific template

```
for(x=0;x<100;x++){  
  console.log(process_sentence(sentences[0]));  
}
```

Generate 1000 random sentences

```
for(x=0;x<1000;x++){  
    console.log(random_sentence());  
}
```

Generate a new slogan in the GUI every second

```
var sloganbot = setInterval(function(){changeSlogan()},1000)  
  
clearInterval(sloganbot)
```

Hack the app to spit out your own slogans - using existing structure

- sentences[8] is simple "#o_just #just_because"

```
phrases['o_just']=["Since when did I",  
    "But you always", "He will rudely"];  
phrases['just_because'] = ["kick a football",  
    "smoke bananas", "sing on the phone"];  
  
for(x=0;x<100;x++){  
    console.log(process_sentence(sentences[8]));  
}
```

Hack the app to spit out your own slogans - using new structure

Design:

- "I like eating mice"

- sentence: “#i_do_like #i_do_like_doing #i_like_doing_with”
- i_do_like: “I like”, “I hate”, “I despise”, “I loathe”, “I love”
- i_do_like_doing: “eating”, “smashing”, “feeding”, “teasing”, “tickling”, “destroying”, “making”, “teaching”
- i_like_doing_with: “cats”, “dogs”, “mice”, “peers”, “Systems”, “Managers”, “Testers”, “Developers”, “code”, “beliefs”, “memories”, “attitudes”

Hack the app to spit out your own slogans - using new structure

Implementation:

```
sentences.push(
    "#i_do_like #i_do_like_doing #i_like_doing_with");
phrases['i_do_like']=
    ["I like", "I hate", "I despise", "I loathe", "I love"];
phrases['i_do_like_doing']=
    ["eating", "smashing", "feeding", "teasing",
    "tickling", "destroying", "making", "teaching"];
phrases['i_like_doing_with']=
    ["cats", "dogs", "mice", "peers", "Systems",
    "Managers", "Testers", "Developers", "code",
    "beliefs", "memories", "attitudes"];

for(x=0;x<100;x++){
    console.log(process_sentence(sentences[sentences.length-1]));
}
```

I T.E.S.T With A.T.T.I.T.U.D.E app

A T-Shirt slogan generating app.

```
I
T.E.S.T
with
A.T.T.I.T.U.D.E
```

Where the acronyms would expand out to:

- T.E.S.T - Tactical, Exploratory, Strategic, Testing
 - A.T.T.I.T.U.D.E - Attack, Tenacious, Task, Interrogate, Technical, Understand, Debrief, Experiment
 - eviltester.com/apps/testwith/version/1/testwith.html
-

Exercise

- visit the application page
- eviltester.com/apps/testwith/version/1/testwith.html
- Explore it and see if you can identify what functionality is available
- Inspect the DOM
- look in the source
- look at the comments

Treat the investigation of the application itself as a ‘game’.

Debrief

- What did we learn?
 - What did you see?
 - What did you make it do?
 - Issues?
-

Exercise: Investigation Hints

- Type in text, then? Clear?
 - Inspect
 - `div(s)` with `id(s)`
 - `[clear]` calls `makeExplodedSentenceForm` what does that do?
 - non-visible `gtrig` - why? what does it do?
 - what uses the canvas?
-

Debrief

- What did we learn?
 - What did you see?
 - What did you make it do?
 - Issues?
-

More About?

- [Render] button
 - [Header] button
-

Keep Going

ZType Bots

These bots run against zty.pe

Crude Bot

```
var zTypeBotAlphaChars = "abcdefghijklmnopqrstuvwxyz";
var zTypeBotCharToType = 0;
ztypebot = setInterval(function() {
    ig.game.shoot(
        zTypeBotAlphaChars.charAt(zTypeBotCharToType));
    zTypeBotCharToType = (zTypeBotCharToType + 1)%26;
}, 100);
```

Remember to stop bot:

```
clearInterval(ztypebot);
```

Good Bot

A Z-Type Bot

```
ztypebot = setInterval(function() {
    if (ig.game.currentTarget == null) {
        for (var t in ig.game.targets) {
            if (ig.game.targets[t].length > 0) {
                ig.game.shoot(t);
                break;
            }
        }
    } else {
        while(ig.game.currentTarget != null){
            ig.game.shoot(
                ig.game.currentTarget.remainingWord.charAt(0));
        }
    }
}, 2);
```

BIO - Alan Richardson

Alan Richardson has more than twenty years of professional IT experience, working as a programmer and at every level of the testing hierarchy from tester through head of testing. He has performed [keynote speeches and tutorials at conferences](#) worldwide. Author of [multiple books on testing and automating](#). Alan also has created [online training courses](#) to help people learn [Technical Web Testing](#) and [Selenium WebDriver with Java](#). He works as an [independent consultant](#), helping companies improve their use of automation, agile, and exploratory technical testing.

Alan Richardson (www)

- www.compendiumdev.co.uk

- www.eviltester.com
 - www.seleniumsimplified.com
 - www.javafortesters.com
 - uk.linkedin.com/in/eviltester
 - testerhq.com - Aggregated Feed
-

Follow

- LinkedIn - [[@eviltester](https://uk.linkedin.com/in/eviltester)](<https://uk.linkedin.com/in/eviltester>)
 - Twitter - [[@eviltester](https://twitter.com/eviltester)](<https://twitter.com/eviltester>)
 - Instagram - [[@eviltester](https://www.instagram.com/eviltester)](<https://www.instagram.com/eviltester>)
 - Facebook - [[@eviltester](https://facebook.com/eviltester/)](<https://facebook.com/eviltester/>)
 - Youtube - [EvilTesterVideos](#)
 - Pinterest - [[@eviltester](https://uk.pinterest.com/eviltester/)](<https://uk.pinterest.com/eviltester/>)
 - Github - [[@eviltester](https://github.com/eviltester/)](<https://github.com/eviltester/>)
 - Slideshare - [[@eviltester](http://www.slideshare.net/eviltester)](www.slideshare.net/eviltester)
-

Books Written By Alan Richardson

- [Java For Testers](#)
 - [Dear Evil Tester](#)
 - [Automating and Testing a REST API](#)
 - [Selenium Simplified](#)
-

Online Training Courses By Alan Richardson

- [Selenium Webdriver With Java](#)
 - [Technical Web Testing 101](#)
 - [Evil Tester Talks: Technical Testing](#)
 - [Case Study: Java Desktop Application Technical Training](#)
-

The End